EvoCodeBench: An Evolving Code Generation Benchmark Aligned with Real-World Code Repositories

Jia Li ♂[1], Ge Li[1], Xuanming Zhang[1], Yihong Dong[1], Zhi Jin[1]
[1]School of Computer Science, Peking University
lijia@stu.pku.edu.cn, lige@pku.edu.cn

Abstract

How to evaluate Large Language Models (LLMs) in code generation is an open question. Existing benchmarks demonstrate poor alignment with real-world code repositories and are insufficient to evaluate the coding abilities of LLMs. This paper proposes a new benchmark - **EvoCodeBench** to address the preceding problems, which has three primary advances. ❶ EvoCodeBench aligns with real-world repositories in multiple dimensions, *e.g.,* code distributions and dependency distributions. ❷ EvoCodeBench offers comprehensive annotations (*e.g.,* requirements, reference code, and reference dependencies), and robust evaluation metrics (*e.g.,* Pass@k and Recall@k). ❸ EvoCodeBench is an evolving benchmark to avoid data leakage. We build an automatic pipeline to update EvoCodeBench from the latest repositories. We release the first version - EvoCodeBench-2403, containing 275 samples from 25 real-world repositories. Based on EvoCodeBench, we propose **repository-level code generation** and evaluate 10 popular LLMs (*e.g.,* gpt-4, gpt-3.5, DeepSeek Coder, StarCoder 2, CodeLLaMa, Gemma, and Qwen 1.5). Our experiments reveal the coding abilities of these LLMs in real-world repositories. **For example, the highest Pass@1 of gpt-4 only is 20.73% in our experiments.** We also analyze failed cases and summarize the shortcomings of existing LLMs in EvoCodeBench. We release EvoCodeBench, all prompts, and LLMs' completions for further community analysis[1].

1 Introduction

Code generation with Large Language Models (LLMs) has attracted lots of researchers' attention (Guo et al., 2024; Rozière et al., 2023; Lozhkov et al., 2024), and some commercial products have been produced, *e.g.,* GitHub Copilot (GitHub, 2023). In practice, human developers typically write the code for code repositories. Thus, evaluating the coding abilities of LLMs in real-world repositories is necessary. We analyze over 1 million functions from 500 real-world repositories (see Section 3) and think a good benchmark should satisfy the following features.

```python
def has_close_elements(numbers, threshold):
  for idx, elem in enumerate(numbers):
    for idx2, elem2 in enumerate(numbers):
      if idx != idx2:
        distance = abs(elem - elem2)
        if distance < threshold:
          return True
  return False
```

(a) A standalone function in HumanEval

```python
# imapclient.IMAPClient.namespace
def namespace(self):
  data = self._command_and_check("namespace")
  parts = []
  for item in parse_response(data):
    (more lines . . )
      for prefix, separator in item:
        if self.folder_encode:
          prefix = decode_utf7(prefix)
        converted.append((prefix, to_unicode)
      parts.append(tuple(converted))
  return Namespace(*parts)
```

(b) A non-standalone function in a real-world project

Figure 1: Examples of standalone and non-standalone functions. Dependencies are highlighted, *i.e.,* yellow: intra-class dependencies, green: intra-file dependencies, and blue: cross-file dependencies.

- **Real-world Repository.** The benchmark should be collected from real-world code repositories (Yu et al., 2023).
- **Real Code Distribution.** Real-world repositories comprise two types of code, *i.e.,* standalone and non-standalone code. As shown in Figure 1, a standalone function solely uses built-in or public libraries, while a non-standalone one contains context-aware *dependencies* (*i.e.,* invocations of code elements defined in current repositories). The benchmark should cover both types of code

[1]https://github.com/seketeam/EvoCodeBench

Table 1: The comparison between existing benchmarks and EvoCodeBench.

Benchmark	Real Repo.	Real Code Distribution	Comprehensive Annota.	Robust Metric	Avoiding Data Leak.
CoNaLA (Yin et al., 2018)	✖	✖	✖	✖	✖
Concode (Iyer et al., 2018)	✔	✖	✖	✖	✖
HumanEval (Chen et al., 2021)	✖	✖	✖	✖	✖
MBPP (Austin et al., 2021)	✖	✖	✖	✖	✖
APPS (Hendrycks et al., 2021)	✖	✖	✖	✖	✖
PandasEval (Zan et al., 2022)	✖	✖	✖	✖	✖
NumpyEval (Zan et al., 2022)	✖	✖	✖	✖	✖
AixBench (Li et al., 2023b)	✔	✖	✖	✖	✖
ClassEval (Du et al., 2023)	✖	✖	✖	✖	✖
CoderEval (Yu et al., 2023)	✔	✖	✖	✔	✖
EvoCodeBench (Ours)	✔	✔	✔	✔	✔

and ensure their ratios are realistic. The number of dependencies should also be consistent with real-world repositories.

- **Comprehensive Annotations.** The benchmark can offer comprehensive annotations, including natural language requirements, original repositories, and ground truths (code and dependencies).

- **Robust Evaluation Metrics.** The benchmark should contain test cases to evaluate models' predictions and report Pass@k. Metrics are also required to assess the abilities of LLMs to generate dependencies.

- **Avoidng Data Leaking.** With more LLMs emerging, the benchmark should avoid potential data leakage (Huang et al., 2023).

However, as shown in Table 1, none of the existing benchmarks satisfies all aforementioned features. The problem hinders the evaluation and development of LLMs in the real development process.

To address the above problem, we propose a new code generation benchmark named EvoCodeBench, which aligns with real-world code repositories. As shown in Table 1, EvoCodeBench satisfies the above features. ❶ EvoCodeBench is collected from high-quality open-source repositories in the real world. ❷ EvoCodeBench is constructed through a rigorous pipeline and aligns with real-world repositories. Specifically, the distributions of code and dependencies in EvoCodeBench are consistent with the ones in 500 real-world repositories. Detailed statistics are in Section 2.4. ❸ EvoCodeBench offers comprehensive annotations, *e.g.,* detailed requirements, original repositories, reference code, and reference dependencies. ❹ EvoCodeBench leverages test cases to check models' predictions and report Pass@k. It also proposes Recall@k to evaluate the dependencies in predictions. ❺ EvoCodeBench is an evolving benchmark and will be dynamically updated every period (*e.g.,* 6 months) to avoid data leakage. In this paper, we release the first version - EvoCodeBench-2403, which consists of 275 samples from 25 real-world repositories. Details of the collection process can be found in Section 3.

Based on EvoCodeBench, we propose **repository-level code generation**, which simulates the developers' coding process in a working repository. The task asks models to write the code based on requirements and a complete repository.

We evaluate 10 popular LLMs (*i.e.,* gpt-4 (OpenAI, 2023b), gpt-3.5 (OpenAI, 2023a), DeepSeek Coder (Guo et al., 2024), StarCoder 2 (Lozhkov et al., 2024), CodeLLaMa (Rozière et al., 2023), Gemma (GemmaTeam, 2024), and Qwen 1.5 (Bai et al., 2023)). These LLMs exhibit low performance on EvoCodeBench, especially compared to their performance on previous benchmarks. **For example, gpt-4-turbo-1106 achieves a Pass@1 score of 80% on HumanEval, while its highest Pass@1 on EvoCodeBench is only 20.73%.** Our results reveal the coding abilities of these LLMs in real-world repositories. We further analyze failed cases and summarize the shortcomings of existing LLMs in EvoCodeBench.

In summary, our contributions are as follows:

- We summarize five features (see Table 1) that a code generation benchmark for real-world repositories should satisfy.

- We propose a new code generation benchmark - EvoCodeBench, satisfying the above features. We released the first version and will continually update it.

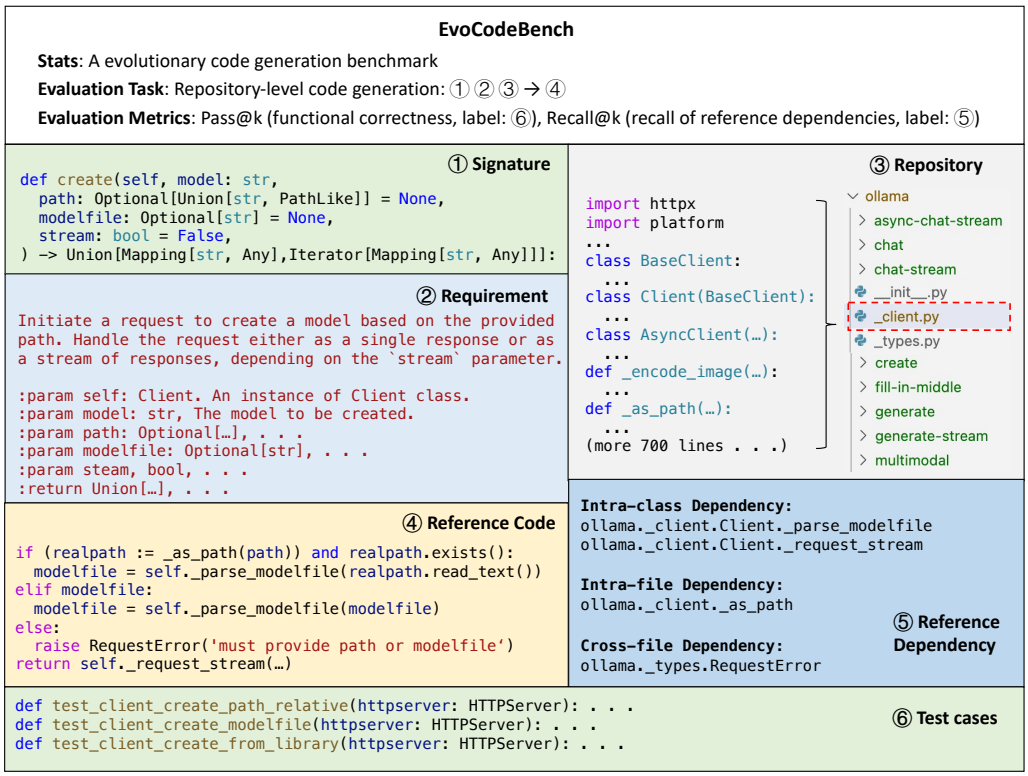

Figure 2: An overview of EvoCodeBench. Each sample consists of six components.

- We propose repository-level code generation, which provides a challenging and realistic evaluation scenario.
- We evaluate 10 popular LLMs on EvoCodeBench, analyzing their strengths and shortcomings in repository-level code generation.

We hope EvoCodeBench can align with the actual experiences of developers during the practical development process. By EvoCodeBench, practitioners can pick up superior LLMs and facilitate the application of code generation techniques in real-world repositories.

2 EvoCodeBench

In this section, we first show an overview of EvoCodeBench and then describe its tasks and metrics. Finally, we present the first version - EvoCodeBench-2403 and its statistics.

2.1 Overview

Figure 2 shows a sample in EvoCodeBench. Each sample consists of six components.

❶ **Function Signature:** The signature of the target code. ❷ **Requirement:** An English description detailing the functionality of the target code. ❸ **Repository:** The current repository contains hundreds of code files. ❹ **Reference Code:** A developer-written implementation of the target code. This code may invoke dependencies defined in the current repository. ❺ **Reference Dependency:** The dependencies invoked in the reference code include intra-class, intra-file, and cross-file dependencies. ❻ **Test Cases:** Test cases are used to check the functional correctness of the code.

2.2 Task Definition

Traditional benchmarks fall into a simple requirement-to-code task. In contrast, EvoCodeBench proposes a more realistic task - **repository-level code generation**. This task simulates the developers' coding process in a working repository. Given a requirement and a repository, LLMs are tasked to generate the code for the repository.

2.3 Evaluation Metrics

Pass@k (Functional Correctness). Following previous studies (Chen et al., 2021; Austin et al., 2021; Yu et al., 2023), we assess the functional correctness of programs by executing test cases and compute the unbiased Pass@k. Specifically, we generate $n \geq k$ programs per requirement, count the

Table 2: The comparison between existing code generation benchmarks and EvoCodeBench-2403. `SA`, `NL`, and `Depend` are the abbreviations of "standalone", "natural language requirements", and "dependency", respectively.

Benchmark	Code Distribution				Dependency Distribution		Annotation	Release Data
	#Repo.	#Sample	SA	Non-SA	#Type	#Avg.		
CoNaLa	–	500	100%	0%	0	0	NL, Code	`2018-5`
HumanEval	–	164	100%	0%	0	0	NL, Code	`2021-7`
MBPP	–	974	100%	0%	0	0	NL, Code	`2021-8`
APPS	–	5,000	100%	0%	0	0	NL, Code	`2021-11`
PandasEval	–	101	100%	0%	0	0	NL, Code	`2022-6`
NumpyEval	–	101	100%	0%	0	0	NL, Code	`2022-6`
AixBench	N/A	175	100%	0%	0	0	NL, Code	`2023-2`
ClassEval	–	100	100%	0%	0	0	NL, Code, Depend. Name	`2023-8`
Concode	N/A	2,000	20%	80%	1	1.23	NL, Code	`2018-8`
CoderEval	43	230	36%	64%	3	1.73	NL, Code, Depend. Name	`2023-2`
EvoCodeBench-2403	25	275	27%	73%	3	3.46	NL, Code, Depend. Path, Repository	`2024-3`
500 Real Repositories	500	1M+	27%	73%	3	3.22	–	–

number of correct programs $c \leq n$ that pass test cases, and calculate the Pass@k:

$$\text{Pass@}k := \mathop{\mathbb{E}}_{\text{Requirements}} \left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}} \right] \quad (1)$$

Recall@k (Recall of Reference Dependency). Besides the functional correctness, we expect LLMs to invoke relevant dependencies defined in contexts. Hence, we propose Recall@k, which gauges the recall of reference dependencies in generated programs.

Specifically, LLMs generate k programs per requirement. For the i-th program, we employ a parser[2] to extract its dependencies as $\mathbb{P}_i$. Subsequently, we compare $\mathbb{P}_i$ with reference dependencies $\mathbb{R}$ and compute the Recall@k:

$$\text{Recall@}k := \mathop{\mathbb{E}}_{\text{Requirements}} \left[\max_{i \in [1,k]} \frac{|\mathbb{R} \cap \mathbb{P}_i|}{|\mathbb{R}|} \right] \quad (2)$$

where $|\cdot|$ means the number of elements of a set.

2.4 EvoCodeBench-2403

This paper releases the first version of EvoCodeBench named EvoCodeBench-2403. The statistics of EvoCodeBench-2403 are shown in Table 2. We discuss its features as follows.

❶ **Alignment with real-world code repositories.** EvoCodeBench-2403 consists of 275 samples collected from 21 real-world repositories. As shown in Table 2, the code distribution of EvoCodeBench-2403 is consistent with that of 500 real-world repositories. The average number of dependencies per program in EvoCodeBench-2403 is also close to that of 500 real-world repositories.

❷ **Comprehensive Annotations.** We provide requirements, reference code, reference dependency and the complete repository for each sample. Previous works (*i.e.,* CoderEval, ClassEval) only provide dependencies' names (*e.g.,* `close`). Because many functions have the same name in practice, it is hard to identify whether generated dependencies are correct by relying on names. EvoCodeBench annotates dependencies with paths (*e.g.,* `A.py::ClassB::close`), addressing ambiguity and biases. These annotations offer a broad arena to explore repository-level code generation and evaluation.

❸ **Latest repositories to avoid data leakage.** Considering the latest LLM's (Lozhkov et al., 2024) training data is up to `2023-9`, EvoCodeBench-2403 is collected from real-world repositories that were created from `2023-10` to `2024-2`. In future versions, we will continually update EvoCodeBench using the latest repositories.

3 Benchmark Collection Pipeline

We build an automatic pipeline for collecting EvoCodeBench from the latest repositories. The pipeline consists of four stages as follows.

❶ **Repository Selection.** We crawl high-quality repositories from GitHub satisfying the following criteria: open-source Python projects; created in recent months; non-fork and non-malicious projects; more than 50 stars; and having explicit unit tests.

❷ **Function Parse.** We extract functions from

[2]We develop the parser based on an open-source static analysis tool - Pyan (Pyan, 2023).

repositories and exclude trivial functions (*e.g.,* empty or initialization functions). We extract each function's signature and function body (*i.e.,* reference code). We developed a static analysis-based parser to extract reference dependencies with the reference code.

❸ **Tests Construction.** For each function, we extract test cases invoking it from current repositories. We use `pip`[3] to automatically install required packages for each repository and leverage `Pytest`[4] to run test cases. Functions without executable test cases are excluded.

❹ **Deduplication.** To improve the diversity of EvoCodeBench, we perform repository-level deduplication based on the Jaccard similarities in code files and imports. The former removes duplicate repositories in text surfaces, and the latter removes repositories in domains that are too similar.

❺ **Requirement Annotation.** Manually writing requirements is time-consuming and laborious. Inspired by the powerful abilities of LLMs in code comment generation (Geng et al., 2024), we leverage LLMs to generate natural language requirements. Specifically, we manually craft a few-shot prompt, which teaches LLMs to write requirements in a specific format (*i.e.,* functional descriptions and input-output parameters). Appendix A.1 the details of prompts.

❻ **Benchmark Construction.** Finally, we select samples from the outputs of step ❺ to construct EvoCodeBench. We strive to make EvoCodeBench satisfy the following goals: consistent with the code distribution observed in 500 real-world repositories; close to the average number of dependencies in 500 real-world repositories; including as many samples as possible.

4 Experiments

4.1 Studied LLMs

Table 3 shows 10 studied LLMs in our experiments. They cover closed-source LLMs (*i.e.,* gpt-4 (OpenAI, 2023b), gpt-3.5 (OpenAI, 2023a)) and open-source LLMs (*i.e.,* DeepSeek Coder (Guo et al., 2024), StarCoder 2 (Lozhkov et al., 2024), CodeLLaMa (Rozière et al., 2023), Gemma (GemmaTeam, 2024), and Qwen 1.5 (Bai et al., 2023)). We use official interfaces or implementations to reproduce these LLMs. The details of LLMs can be found in Appendix B.1.

[3] https://pypi.org/project/pip/
[4] https://docs.pytest.org/en/8.0.x/

Table 3: Studied LLMs in this paper. Context L.: Context Window.

Type	Name	Version	Context W.
Closed-source	gpt-4	gpt-4-turbo-1106	128,000
	gpt-3.5	gpt-3.5-turbo-1106	16,385
Open-source	StarCoder 2	15B	16,384
	StarCoder 2	7B	16,384
	DeepSeek Coder	33B	16,384
	DeepSeek Coder	6.7B	16,384
	CodeLLaMa	13B	16,384
	CodeLLaMa	7B	16,384
	Gemma	7B	8,192
	Qwen 1.5	7B	32,768

4.2 Experimental Setting

Repository-level code generation takes a requirement and a repository as inputs. Typically, a repository consists of hundreds of code files and is very long. For example, the average length of 500 real-world repositories is 1.1 million tokens, surpassing the context windows of existing LLMs (*e.g.,* gpt-4: 128k tokens). Inspired by related work (Shrivastava et al., 2023), we try to extract parts of code contexts from the repository as inputs and design the following experimental settings.

❶ **Without context.** In this setting, we ignore contexts and directly generate the code based on requirements and signatures.

❷ **Local File (Completion).** The local file denotes the code file where the reference code is in. This setting simulates the scenario where developers continue to write code at the end of a file. Thus, we consider code snippets above the reference code in the local file as contexts. Then, LLMs generate code in an autoregressive manner based on requirements, signatures, and contexts.

❸ **Local File (Infilling).** Different from the Local File (Completion) setting, this setting simulates the scenario where developers infill code in the middle of a file. Thus, we use the code snippets above and below the reference code in the local file as contexts. We evaluate LLMs that support code infilling and construct input sequences using official formats.

The prompt templates used in the above settings are shown in Appendix B.2. We note that there are other approaches to extracting relevant contexts. We consider this to be beyond the scope of this paper and leave the exploration to future work.

4.3 Evaluation

We use Pass@k and Recall@k (see Section 2.3) to assess generated programs. In this paper, $k \in$

Table 4: Pass@k and Recall@k of LLMs on EvoCodeBench-2403. Bold and underlined data indicate top-1 and top-2 results, respectively.

LLMs	Size	Pass@1	Pass@3	Pass@5	Pass@10	Recall@1	Recall@3	Recall@5	Recall@10
Local File (Infilling)									
gpt-4	N/A	**20.73**	**23.03**	**24.11**	**25.34**	68.24	70.63	72.05	73.52
gpt-3.5	N/A	17.82	21.78	23.06	24.46	61.94	68.13	69.69	70.85
DeepSeek Coder	33B	19.64	22.78	24.29	26.01	**71.46**	**79.93**	**82.11**	**86.25**
DeepSeek Coder	6.7B	17.82	21.02	22.40	23.97	69.58	74.04	78.00	83.22
StarCoder 2	15B	14.91	17.54	18.63	19.86	50.90	53.29	55.89	61.76
StarCoder 2	7B	15.27	17.29	18.63	20.09	56.35	60.59	63.74	74.20
Local File (Completion)									
gpt-4	N/A	**17.45**	**19.65**	**20.80**	**22.41**	63.49	68.67	70.00	72.07
gpt-3.5	N/A	15.64	17.29	18.21	19.36	61.44	66.25	66.82	69.89
DeepSeek Coder	33B	14.18	17.57	18.66	19.95	66.90	**72.83**	74.40	80.02
DeepSeek Coder	6.7B	13.45	17.10	18.81	21.07	65.76	72.32	75.61	78.45
StarCoder 2	15B	13.45	15.44	17.84	19.59	**68.55**	71.37	74.76	77.70
StarCoder 2	7B	13.82	15.15	16.18	17.65	62.93	69.85	73.54	78.40
CodeLLaMa	13B	12.73	15.78	16.86	18.19	63.34	71.26	**76.43**	80.11
CodeLLaMa	7B	12.73	15.33	16.00	16.93	63.33	69.79	71.91	76.50
Gemma	7B	10.55	13.25	14.31	15.48	58.02	70.57	74.44	**80.76**
Qwen 1.5	7B	5.45	7.04	7.91	9.07	39.21	44.02	50.17	58.42
Without Contexts									
gpt-4	N/A	**7.27**	**10.05**	**10.70**	11.49	21.58	23.93	25.69	26.23
gpt-3.5	N/A	6.55	7.85	8.28	8.73	21.66	24.31	24.77	25.40
DeepSeek Coder	33B	6.91	8.92	9.79	11.03	**27.67**	**32.73**	34.92	37.76
DeepSeek Coder	6.7B	5.82	8.56	9.67	11.26	25.89	32.06	**35.59**	**38.33**
StarCoder 2	15B	6.18	8.77	9.95	**11.53**	24.03	29.86	33.62	36.91
StarCoder 2	7B	5.82	6.72	7.43	8.62	27.39	32.60	34.88	36.81
CodeLLaMa	13B	5.45	7.38	8.37	9.95	25.52	31.28	33.66	36.36
CodeLLaMa	7B	5.45	6.94	7.75	9.03	26.97	31.17	34.08	36.82
Gemma	7B	6.18	6.86	7.64	8.66	21.84	29.98	33.61	35.23
Qwen 1.5	7B	4.00	4.72	5.38	6.18	16.33	13.56	16.34	21.06

$[1, 3, 5, 10]$. When $k = 1$, we use the greedy search and generate a single program per requirement. When $k > 1$, we use the nucleus sampling with a temperature 0.4 and sample 20 programs per requirement. We set the top-p to 0.95 and the max generation length to 500.

Because EvoCodeBench is an evolving benchmark, this paper evaluates LLMs upon EvoCodeBench-2403. Note that the Pass@k and Recall@k between different versions of EvoCodeBench are not comparable.

4.4 Main Results

The Pass@k and Recall@k in three experimental settings are shown in Table 7.

Without Context. gpt-4 achieves the highest Pass@k among all LLMs. However, compared to previous benchmarks, these LLMs' performance in EvoCodeBench-2403 drops dramatically. For example, gpt-4 achieves a Pass@1 score of 88.4 on HumanEval, while it scores 7.27 on Pass@1 upon EvoCodeBench-2403. On the one hand, the decreases validate our motivation that existing benchmarks can not comprehensively assess the coding abilities of LLMs in practical projects. On the other hand, the results emphasize the importance of contexts in repositories. Without the necessary context, LLMs lack the domain knowledge of current repositories and generate the wrong programs.

Local File (Completion) and (Infilling). After introducing the contexts within local files, the Pass@k and Recall@k of all LLMs obviously increase. For example, the Pass@1 of gpt-4 is improved by 104% and 152% in two settings, respectively. We attribute the improvements to the domain knowledge contained in contexts. Figure 3 shows a uniquely successful case in the Local File (Completion) setting. The key to writing this function is to know cache directories. Without context, gpt-4 fabricated a non-existent field as cache direc-

Contexts (above):

```
import concurrent
import json
                                . . .
def _get_num_nodes() -> int:
                                . . .
def _get_cache_dir(name: Optional[str] = None):
                                . . .
def _get_cache_data_dir(name: Optional[str] = None):
                                . . .
```

Input Code:

```
def _cleanup_cache(self) -> None:
  """ Clean up cache directories by removing them if
they exist to prevent issues from corrupted files
from previous runs, and then recreates these
directories to ensure they are available for use.
  :param self: DataProcessor. ...
  :return: No return values.
  """
```

gpt-4-turbo-1106 (without context):

```
for cache_dir in self.cache_directories:
# Check if the cache directory exists
  if os.path.exists(cache_dir):
    . . .
```

gpt-4-turbo-1106 (local completion):

```
# Define the cache directories
cache_data_dir = _get_cache_data_dir()
cache_chunks_dir = _get_cache_dir()

# Remove the cache data directory if it exists
if os.path.exists(cache_data_dir):
  . . .
```

Figure 3: A uniquely successful case in Local File (Completion) setting.

tories, generating the incorrect code. In fact, two functions for returning the cache directories are available in the local file. After introducing the local file, gpt-4 successfully gets cache directories by invoking these functions and generates the correct code.

Error Analyses. Although promising, the Pass@k of existing LLMs is still low and far from practical applications. To determine LLMs' shortcomings, we manually analyze 50 error cases of gpt-4 in the Local File (Infilling) setting. We found that most of the cases (29 cases) failed due to implementation logic errors. 20 cases failed since the necessary contexts were missing, *e.g.,* APIs defined in other files. Besides, one case failed because of the vague requirement. It shows that existing LLMs' reasoning and coding abilities need to be improved. Meanwhile, how to utilize more contexts is necessary to explore.

We also obtain some interesting findings from Table 7.

❶ **LLMs successfully generate some dependencies without context.** Theoretically, LLMs do not see the contexts and cannot generate dependencies. According to Table 7, we are surprised to find that LLMs can generate some dependencies without context. We manually inspect successful cases and summarize two reasons. First, LLMs can reason about some easy dependencies from requirements, *e.g.,* initialization functions of returned objects. Second, LLMs can "guess" dependencies from their functionalities. In practice, dependencies' names are relevant to their functional descriptions, *e.g.,* `send_request()` means send a request to the server. LLMs are trained with a large code corpus and can learn the naming conventions. Thus, LLMs may successfully guess some dependencies from their functionalities.

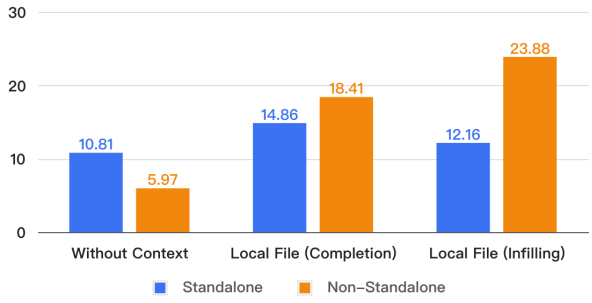

Figure 4: Pass@1 of gpt-4 on different program types.

❷ **More contexts benefit code generation.** Based on Table 7, we compare the performance of an LLM (*e.g.,* gpt-4) under different settings. Obviously, the more input contexts, the better the performance of the LLM. It inspires practitioners to extend the context windows of LLMs and input more contexts.

❸ **The gpt family models have higher Pass@k and lower Recall@k, while other models are the opposite.** We speculate the reason is that gpt family models are instruction-tuned models and focus on performing tasks based on given instructions. With limited contexts, gpt family models are conservative and tend to generate code independently. Other LLMs are standard language models trained with real code files containing dependencies. They are aggressive and generate dependencies that may exist. The comparisons show the importance of instruction tuning in practical applications.

4.5 Empirical Leassons

Based on the above experiments, we summarize the empirical lessons we learned as

❶ EvoCodeBench pose new challenges, *i.e.,* repository-level code generation. The performance of existing LLMs on EvoCodeBench-2403 drops dramatically compared to their performance on pre-

Table 5: The comparison between auto-generated requirements and human-written requirements.

Annotator	Win / Tie / Lose	Cost (Time)	Cost (Money)
gpt-4	5 / 41 / 4	12m30s	$0.54
Human	4 / 41 / 5	4h10m	$31.25

vious benchmarks.

❷ LLMs benefit from code contexts in current repositories. With limited context windows, the contexts from local files can improve gpt-4 by 152% in Pass@1.

❸ The main reasons why programs generated by LLMs fail are logic errors and incomplete contexts. How to enhance reasoning abilities and context windows of LLMs is important.

5 Discussion

Evaluation of auto-generated requirements. We leverage an LLM (gpt-4 in this paper) to generate natural language requirements for functions automatically. To assess the quality of auto-generated requirements, we randomly select 50 functions from EvoCodeBench-2403 and compare requirements from gpt-4 and human developers. We hire two developers to write the requirements and two others to evaluate auto-generated and human-written requirements. The evaluation metrics include *completeness* (whether the requirements cover the intent of code), *clarity* (whether the requirements are clear and user-friendly). All developers are paid according to the relevant policies[5] ($7.5 per hour).

The evaluation results are shown in Table 5. The Cohen's Kappa coefficient between the two evaluators is 0.92. On 41 functions, gpt-4 and developers tie. In the remaining functions, gpt-4 wins by 5 functions, and developers win by 4 functions. These results show that gpt-4 can produce high-quality requirements comparable to human-written requirements in most cases (92% = 46/50). We also inspect the four functions lost by gpt-4 and find that some necessary details (*e.g.,* hyper-parameters) are missing in its requirements. In the future, we will explore new techniques to solve this problem, *e.g.,* controllable text generation (Dekoninck et al., 2023). Besides the high-quality requirements, gpt-4 shows advantages in costs. As shown in Table 5, got-4 costs less time and money to annotate requirements. Thus, it is a feasible and efficient approach for us to use gpt-4 to annotate requirements for EvoCodeBench.

[5] https://www.worker.gov/

Table 6: Pass@1 and Recall@1 with retrieval-augmented generation.

LLMs	Setting	Pass@1	Recall@1
gpt-4	Without Context	8.31	21.08
	Similar Functions	12.29	45.14
gpt-3.5	Without Context	6.64	21.16
	Similar Functions	11.62	41.93

Retrieval-Augmented Generation (RAG). RAG is to enhance generative models with retrieved information and has achieved promising results in code generation (Li et al., 2023b,c). We try to apply RAG to repository-level code generation and consider the repository to be a retrieval corpus. Because most programs in repositories are not equipped with documentation, we retrieve top-k (*i.e.,* $k = 5$ in this paper) functions with similar names to the target function. Specifically, we split names into tokens based on underscore or camel-case formatting and then match the tokens of names. Finally, we use similar functions as contexts in prompts and further generate code. The experimental results are shown in Table 6. The performance of both LLMs is improved after introducing similar functions. LLMs can know relevant algorithms and dependencies from similar functions, which benefit writing new programs. In the future, we will explore more advanced RAG techniques to improve repository-level code generation.

Results on different program types. Figure 4 shows Pass@1 of gpt-4 on different program types (*i.e.,* standalone and non-standalone). The results are consistent with the above Table 7. Code contexts significantly improve the Pass@1 on non-standalone functions. Meanwhile, the Pass@1 on standalone functions also slightly increases. We speculate that the domain knowledge (*e.g.,* private objects) within contexts helps LLMs understand requirements. However, the Pass@1 on both types of programs is still low. The coding abilities of existing LLMs in real-world repositories need to be further improved.

Results on different dependency types. Figure 5 shows the Recall@1 of gpt-4 on different dependency types (*i.e.,* intra-class, intra-file, and cross-file). The results yield two insights. ❶ Without context, LLMs can reason about some simple dependencies from requirements (*e.g.,* initialization

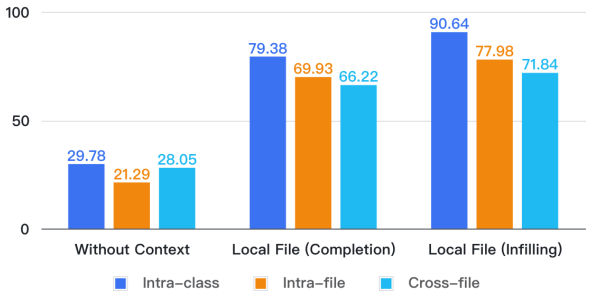

Figure 5: Recall@1 of gpt-4 on different dependency types.

functions of returned objects), but still exhibit low Recall@1 values across three dependency types. ② Local files contain code implementations of intra-class and intra-file dependencies and thus improve the Recall@1 on both types of dependencies. It is surprising that Recall@1 on cross-file dependencies also increases. We inspect successful cases and find that LLMs *copy* cross-file dependencies from relevant programs in local files. However, copied cross-dependencies may be inconsistent with the current code, *e.g.,* inconsistent arguments. It results in gpt-4 with higher Recall@1 but lower Pass@1. Thus, it is necessary for LLMs to see more contexts about cross-file dependencies.

The bias of Recall@k. As stated in Section 2.3, we develop a static analysis-based parser to extract dependencies in generated programs automatically. Because Python is a dynamically typed language, certain dependencies are only determined at runtime and may elude our parser. It may lead to lower Recall@k than actual values.

To gauge the above bias, we randomly select 50 programs generated by gpt-4 and annotate dependencies with them by our parser and two human developers, respectively. Based on the human-annotated and auto-extracted dependencies, we compute two Recall@1 values. The bias of two Recall@1 values is 0.16. Compared to the average variations between LLMs (7.77 in Table 7), 0.16 is slight. Consequently, we believe that Recall@k can effectively rank different LLMs, notwithstanding its slight bias.

6 Related Work

Large Language Models for Code Generation. The rise of pre-training technology has brought new impetus to the field of code generation, both in academia and industry (Li et al., 2022; Shen et al., 2022; Nijkamp et al., 2023; Fried et al., 2023). In this context, more and more LLMs have emerged, achieving significant advancements in code generation, such as Codex (Chen et al., 2021), ChatGPT (OpenAI, 2023a), CodeLlama (Rozière et al., 2023), DeepSeek Coder (Guo et al., 2024), and StarCoder2 (Lozhkov et al., 2024).

To effectively steer LLMs in various code generation scenarios, some works focus on improving the prompt technologies by introducing specific patterns, *e.g.,* Structured Chain-of-Thought (Li et al., 2023a), Self-planning (Jiang et al., 2023), Self-debug (Chen et al., 2023), Self-collaboration (Dong et al., 2023), and AceCoder (Li et al., 2023c).

Code Generation Benchmarks. Early code generation benchmarks (Yin et al., 2018; Chen et al., 2021; Austin et al., 2021; Zan et al., 2022) evaluate code generation on relatively Python functions, such as HumanEval (Chen et al., 2021) and MBPP (Austin et al., 2021). APPS (Hendrycks et al., 2021) evaluates code generation on more difficult competition-style problems. ClassEval (Du et al., 2023) evaluates LLMs on class-level code generation and contains 100 human-crafted self-contained Python classes. Concode (Iyer et al., 2018) and CoderEval (Yu et al., 2023) further introduce non-standalone programs.

We release EvoCodeBench to extend code generation benchmarks. Compared to existing benchmarks, EvoCodeBench aligns with real-world code repositories (*e.g.,* the distributions of code and dependency) and contains more comprehensive annotations (*e.g.,* reference dependencies). Besides, EvoCodeBench is an evolving benchmark and is dynamically updated to address data leakage.

We have also noticed that some benchmarks have recently been proposed for repository-level tasks. CrossCodeEval (Ding et al., 2023), RepoBench (Liu et al., 2023), and RepoEval (Zhang et al., 2023) are code completion benchmarks. They lack the necessary annotations (*e.g.,* natural language requirements) for code generation. SWE-bench (Jimenez et al., 2023) focuses on repairing repositories' issues by revising existing programs. In contrast, EvoCodeBench is collected for code generation and aims to generate new programs based on requirements for a repository. EvoCodeBench offers comprehensive annotations (*e.g.,* natural language requirements, original repositories, reference code, and reference dependencies).

7 Conclusion and Future Work

In this paper, we propose a new code generation benchmark named EvoCodeBench. Collected through a meticulous pipeline, EvoCodeBench aligns with real-world code repositories in multiple dimensions, *e.g.,* code distributions and dependency distributions. Besides, EvoCodeBench is an evolving benchmark and will be dynamically updated every period (*e.g.,* 6 months). Based on EvoCodeBench, we propose repository-level code generation and evaluate 10 popular LLMs. The results reveal the strengths and weaknesses of LLMs in real-world repositories. Compared to previous benchmarks, EvoCodeBench offers a more challenging and realistic evaluation scenario. We hope EvoCodeBench can facilitate the applications of LLMs in real-world repositories.

In the future, we will continue to update EvoCodeBench, *e.g.,* multilingual samples. Besides, we will explore how to improve the performance of LLMs in repository-level code generation, *e.g.,* retrieval-augmented and tool-augmented generation.

8 Limitations

We believe that EvoCodeBench itself has four limitations. ❶ EvoCodeBench is a monolingual benchmark (*i.e.,* requirements in English and code in Python) and ignores other languages. In practice, LLMs require understanding requirements in different natural languages (*e.g.,* Chinese, Spanish) and generating programs in various programming languages (*e.g.,* Java, C). Thus, we plan to build a multilingual EvoCodeBench in future work. ❷ Auto-generated requirements can be improved. As stated in Section 5, auto-generated requirements are comparable to human-written requirements but may lack necessary details. In the future, we will leverage more advanced generation strategies (*e.g.,* controlled text generation (Dekoninck et al., 2023)) to generate requirements.

❸ As stated in Section 5, Recall@k values in EvoCodeBench may have slight biases, *i.e.,* they may be slightly less than actual values. Because Python is a dynamically typed language, certain dependencies can only be identified at runtime and may elude our parser. To gauge the bias introduced by our parser, we manually annotate dependencies within 50 programs generated by gpt-4. Simultaneously, we employ the parser to extract dependencies in the same 50 programs. Based on the human-annotated and auto-extracted dependencies, we compute two Recall@1 values. The bias of two Recall@1 is 0.16. Compared to the average variations between LLMs (7.77 in Table 7), 0.16 is slight. Consequently, the Recall@k can effectively rank different LLMs, notwithstanding its slight bias.

Besides, our evaluation experiments can be further improved in three aspects. ❹ More LLMs. Due to the limited computing budgets, we mainly evaluate 10 mainstream LLMs. It is worth evaluating LLMs with different sizes and fine-tuned models. ❺ More investigations of contexts. As shown in Table 7, we design two straightforward approaches to extracting code contexts from repositories. In the future, we will introduce cross-file contexts and further explore how to utilize contexts effectively. ❻ Tuning hyper-parameters. It is known that LLMs are sensitive to sampling hyper-parameters and prompts. We ensure all LLMs are evaluated under the same experimental settings. Due to the limited computing budgets, we do not carefully tune hyper-parameters and prompts. Thus, there may be better hyper-parameters and prompts to improve the performance of LLMs further.

9 Ethics Consideration

EvoCodeBench is collected from open-source repositories from the real world. We manually check all samples in EvoCodeBench. We ensure all samples do not contain private information or offensive content. We ensure all programs in EvoCodeBench are behaving normally and exclude any malicious programs.

References

Jacob Austin, Augustus Odena, Maxwell I. Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie J. Cai, Michael Terry, Quoc V. Le, and Charles Sutton. 2021. Program synthesis with large language models. *CoRR*, abs/2108.07732.

Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, Binyuan Hui, Luo Ji, Mei Li, Junyang Lin, Runji Lin, Dayiheng Liu, Gao Liu, Chengqiang Lu, Keming Lu, Jianxin Ma, Rui Men, Xingzhang Ren, Xuancheng Ren, Chuanqi Tan, Sinan Tan, Jianhong Tu, Peng Wang, Shijie Wang, Wei Wang, Shengguang Wu, Benfeng Xu, Jin Xu, An Yang, Hao Yang, Jian Yang, Shusheng Yang, Yang Yao, Bowen Yu, Hongyi Yuan, Zheng Yuan, Jianwei Zhang, Xingxuan Zhang, Yichang Zhang, Zhenru Zhang, Chang Zhou, Jingren Zhou, Xiaohuan Zhou, and Tianhang Zhu. 2023. Qwen technical report. *CoRR*, abs/2309.16609.

Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott M. Lundberg, Harsha Nori, Hamid Palangi, Marco Túlio Ribeiro, and Yi Zhang. 2023. Sparks of artificial general intelligence: Early experiments with GPT-4. *CoRR*, abs/2303.12712.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Pondé de Oliveira Pinto, Jared Kaplan, Harrison Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. *CoRR*.

Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. 2023. Teaching large language models to self-debug. *CoRR*, abs/2304.05128.

Jasper Dekoninck, Marc Fischer, Luca Beurer-Kellner, and Martin T. Vechev. 2023. Controlled text generation via language model arithmetic. *CoRR*, abs/2311.14479.

Yangruibo Ding, Zijian Wang, Wasi Uddin Ahmad, Hantian Ding, Ming Tan, Nihal Jain, Murali Krishna Ramanathan, Ramesh Nallapati, Parminder Bhatia, Dan Roth, and Bing Xiang. 2023. Crosscodeeval: A diverse and multilingual benchmark for cross-file code completion. *CoRR*, abs/2310.11248.

Yihong Dong, Xue Jiang, Zhi Jin, and Ge Li. 2023. Self-collaboration code generation via chatgpt. *CoRR*, abs/2304.07590.

Xueying Du, Mingwei Liu, Kaixin Wang, Hanlin Wang, Junwei Liu, Yixuan Chen, Jiayi Feng, Chaofeng Sha, Xin Peng, and Yiling Lou. 2023. Classeval: A manually-crafted benchmark for evaluating llms on class-level code generation. *CoRR*, abs/2308.01861.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida Wang, Eric Wallace, Freda Shi, Ruiqi Zhong, Scott Yih, Luke Zettlemoyer, and Mike Lewis. 2023. Incoder: A generative model for code infilling and synthesis. In *ICLR*. OpenReview.net.

Deep Ganguli, Liane Lovitt, Jackson Kernion, Amanda Askell, Yuntao Bai, Saurav Kadavath, Ben Mann, Ethan Perez, Nicholas Schiefer, Kamal Ndousse, Andy Jones, Sam Bowman, Anna Chen, Tom Conerly, Nova DasSarma, Dawn Drain, Nelson Elhage, Sheer El Showk, Stanislav Fort, Zac Hatfield-Dodds, Tom Henighan, Danny Hernandez, Tristan Hume, Josh Jacobson, Scott Johnston, Shauna Kravec, Catherine Olsson, Sam Ringer, Eli Tran-Johnson, Dario Amodei, Tom Brown, Nicholas Joseph, Sam McCandlish, Chris Olah, Jared Kaplan, and Jack Clark. 2022. Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned. *CoRR*, abs/2209.07858.

GemmaTeam. 2024. Gemma: Open models based on gemini research and technology. *CoRR*, abs/2403.08295.

Mingyang Geng, Shangwen Wang, Dezun Dong, Haotian Wang, Ge Li, Zhi Jin, Xiaoguang Mao, and Xiangke Liao. 2024. Large language models are few-shot summarizers: Multi-intent comment generation via in-context learning. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*, pages 39:1–39:13. ACM.

GitHub. 2023. Github copilot. `https://github.com/features/copilot`.

Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y. K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. Deepseek-coder: When the large language model meets programming - the rise of code intelligence. *CoRR*, abs/2401.14196.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. 2021. Measuring coding challenge competence with APPS. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, NeurIPS Datasets and Benchmarks 2021, December 2021, virtual*.

Yiming Huang, Zhenghao Lin, Xiao Liu, Yeyun Gong, Shuai Lu, Fangyu Lei, Yaobo Liang, Yelong Shen, Chen Lin, Nan Duan, et al. 2023. Competition-level problems are effective llm evaluators. *arXiv preprint arXiv:2312.02143*.

Srinivasan Iyer, Ioannis Konstas, Alvin Cheung, and Luke Zettlemoyer. 2018. Mapping language to code in programmatic context. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, October 31 - November 4, 2018*, pages 1643–1652. Association for Computational Linguistics.

Xue Jiang, Yihong Dong, Lecheng Wang, Qiwei Shang, and Ge Li. 2023. Self-planning code generation with large language model. *CoRR*, abs/2303.06689.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2023. Swe-bench: Can language models resolve real-world github issues? *CoRR*, abs/2310.06770.

Jia Li, Ge Li, Yongmin Li, and Zhi Jin. 2023a. Structured chain-of-thought prompting for code generation. *arXiv preprint arXiv:2305.06599*.

Jia Li, Yongmin Li, Ge Li, Zhi Jin, Yiyang Hao, and Xing Hu. 2023b. Skcoder: A sketch-based approach for automatic code generation. In *45th IEEE/ACM International Conference on Software Engineering, ICSE 2023, Melbourne, Australia, May 14-20, 2023*, pages 2124–2135. IEEE.

Jia Li, Yunfei Zhao, Li Yongmin, Ge Li, and Zhi Jin. 2023c. Acecoder: Utilizing existing code to enhance code generation. *arXiv preprint arXiv:2303.17780*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. 2022. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097.

Tianyang Liu, Canwen Xu, and Julian J. McAuley. 2023. Repobench: Benchmarking repository-level code auto-completion systems. *CoRR*, abs/2306.03091.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. 2024. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*.

Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. 2023. Codegen: An open large language model for code with multi-turn program synthesis. In *ICLR*. OpenReview.net.

OpenAI. 2023a. gpt-3.5-turbo. https://platform.openai.com/docs/models/gpt-3-5.

OpenAI. 2023b. GPT-4 technical report. *CoRR*, abs/2303.08774.

Pyan. 2023. Pyan. https://github.com/davidfraser/pyan.

Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Joanna Bitton, Manish Bhatt, Cristian Canton-Ferrer, Aaron Grattafiori, Wenhan Xiong, Alexandre Défossez, Jade Copet, Faisal Azhar, Hugo Touvron, Louis Martin, Nicolas Usunier, Thomas Scialom, and Gabriel Synnaeve. 2023. Code llama: Open foundation models for code. *CoRR*, abs/2308.12950.

Sijie Shen, Xiang Zhu, Yihong Dong, Qizhi Guo, Yankun Zhen, and Ge Li. 2022. Incorporating domain knowledge through task augmentation for front-end javascript code generation. In *ESEC/SIGSOFT FSE*, pages 1533–1543. ACM.

Disha Shrivastava, Hugo Larochelle, and Daniel Tarlow. 2023. Repository-level prompt generation for large language models of code. In *International Conference on Machine Learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA*, volume 202 of *Proceedings of Machine Learning Research*, pages 31693–31715. PMLR.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurélien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *CoRR*, abs/2302.13971.

Pengcheng Yin, Bowen Deng, Edgar Chen, Bogdan Vasilescu, and Graham Neubig. 2018. Learning to mine aligned code and natural language pairs from stack overflow. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR 2018, Gothenburg, Sweden, May 28-29, 2018*, pages 476–486. ACM.

Hao Yu, Bo Shen, Dezhi Ran, Jiaxin Zhang, Qi Zhang, Yuchi Ma, Guangtai Liang, Ying Li, Tao Xie, and Qianxiang Wang. 2023. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. *CoRR*, abs/2302.00288.

Daoguang Zan, Bei Chen, Dejian Yang, Zeqi Lin, Minsu Kim, Bei Guan, Yongji Wang, Weizhu Chen, and Jian-Guang Lou. 2022. CERT: continual pre-training on sketches for library-oriented code generation. In *Proceedings of the Thirty-First International Joint Conference on Artificial Intelligence, IJCAI 2022, Vienna, Austria, 23-29 July 2022*, pages 2369–2375. ijcai.org.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. 2023. Repocoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 2471–2484. Association for Computational Linguistics.

Please write natural language comments for the given Python function.

The template of natural language comments is shown as follows:
1. [Required] What does this function do.
2. [Required] Input-Output parameters
:param arg1: data type [optional], what it is [required], how it is used [optional].
:param arg2: ....
:return: data type [optional], what it is [required]. Or No return values.

Here is an example:
Input Code:
```Python
{example_code}
```
Requirement:
```
{example_requirement}
```

Input Code:
```Python
{input_code}
```
Requirement:

Figure 6: The prompt template for generating requirements with gpt-4.

A Benchmark Collection Details

A.1 Requirement Annotation

The prompt template for generating requirements is shown in Figure 6. `{example_code}`, `{example_requirement}`, and `{input_code}` are placeholders. We manually write a function with requirements as the demonstration example. Then, we fill `{input_code}` with functions and leverage gpt-4 to generate the corresponding requirements. We use the greedy search and generate a requirement for each function.

A.2 Projects in EvoCodeBench-2403

B Experimental Details

B.1 Base LLMs

In this paper, we select 6 popular LLMs as base LLMs and evaluate them on EvoCodeBench-2403. The details of these LLMs are described as follows.

- **gpt-4** (OpenAI, 2023b), released by OpenAI on March 14, 2023, marks another milestone in the field of natural language processing. gpt-4 demonstrates superior performance compared to previous gpt models (Bubeck et al., 2023). In our experiments, we use the version - gpt-4-1106. Its training data up to April 2023. It continues the auto-regressive prediction of the next token training objective inherited from the GPT series models and incorporates reinforcement learning with human feedback (RLHF) and red-teaming (Ganguli et al., 2022) techniques. However, the pre-training data scope and scale, model size, and parameters remain closed-source at present.

- **gpt-3.5-turbo** (OpenAI, 2023a) is an improved gpt-3 model enhanced by a three-stage reinforcement learning with human feedback (RLHF) algorithm. Apart from improving instruction-following capabilities, the RLHF algorithm proves highly effective in mitigating the generation of harmful or toxic content, which is crucial for the practical deployment of LLMs in security-sensitive contexts. we utilized the released versions of gpt-3.5, namely gpt-3.5-turbo-1106, with training data up to September 2021. However, similar to gpt-4, the training details, training data, and model weights are currently closed-source.

- **CodeLLaMa** (Rozière et al., 2023), based on the LLama2 architecture by Meta-AI[6], was fine-tuned and open-sourced by the company on August 25, 2023, with versions of 7B, 13B, and 34B. A 70B version was released on January 30, 2024 (Rozière et al., 2023). CodeLLama is primarily trained on nearly deduplicated publicly available code datasets. The first three models were trained on 500 billion tokenized code, while the latest 70B model was trained on 1T tokens. Similar to the LLaMa series, CodeLLaMa also follows a decoder-only architecture. We evaluated CodeLLaMa-Python-{7B, 13B} upon our EvoCodeBench.

- **DeepSeek Coder** (Guo et al., 2024) is a large language model for programming tasks released by DeepSeek-AI[7] in November 2, 2023. DeepSeek Coder consists of a series of code language models, each trained from scratch on 2T tokens, containing 87% code and 13% natural language. DeepSeek Coder provides code models with 1.3B, 6.7B and 33B parameter sizes. In terms of model architecture, each model integrates decoder-only Transformer, incorporating Rotary Position Embedding and FlashAttention v2. We evaluated DeepSeek Coder-{6.7B, 33B} on our EvoCodeBench.

- **StarCoder 2** (Lozhkov et al., 2024) was re-

[6] https://ai.meta.com/
[7] https://www.deepseek.com/

Table 7: Projects detail on EvoCodeBench-2403.

Repository	Created	Stars	Py Files	Py Lines	Samples	Domin
Test-Agent	2023-10-20	440	85	15278	1	Deep Learning
skfolio	2023-12-14	813	158	33852	13	Statistical Learning
camp_zipnerf	2024-01-19	523	53	18973	54	Image Processing
microagents	2023-12-11	674	45	2918	18	Deep Learning
open-iris	2023-12-09	161	140	13933	14	Image Processing
litdata	2024-02-15	114	56	11713	59	Data Science
nlm-ingestor	2024-01-17	643	56	16674	4	Internet
AutoRAG	2024-01-10	259	115	7735	13	Data Science
XAgent	2023-10-16	7054	148	17623	3	Deep Learning
tanuki_py	2023-10-16	606	108	10146	9	Text Processing
UHGEval	2023-11-06	148	34	2938	3	Data Science
Generalizable-BEV	2023-10-30	136	570	132407	8	Image Processing
EasyVolcap	2023-12-07	442	308	51723	20	Image Processing
UniRef	2023-12-22	208	382	70042	23	Image Processing
contrastors	2024-01-30	346	62	13774	1	Deep Learning
gaussian-splatting-lightning	2023-10-06	168	76	9935	1	Image Processing
scepter	2023-12-21	190	244	41519	1	Deep Learning
microsearch	2024-02-05	336	5	231	2	Internet
ollama-python	2023-12-09	898	13	2089	12	Deep Learning
Python-Type-Challenges	2023-10-23	343	121	3208	1	Education
stable-fast	2023-10-17	871	82	11948	2	Deep Learning
stable-diffusion-webui-forge	2024-01-14	2537	1112	210946	3	Deep Learning
openlogprobs	2023-11-22	174	6	524	1	Deep Learning
searcharray	2023-11-03	133	25	4217	6	Data Science
deluder	2023-12-01	115	34	1894	3	Internet

leased by BigCode[8] on December 8, 2023 with 3 different parameters, 3B, 7B and 15B. StarCoder2 is trained on The Stack v2, a new large-scale high-quality code dataset. All models were trained using Grouped Query Attention, a contextual window of 16,384 tokens with a sliding window attention of 4,096 tokens, using the Fill-in-the-Middle objective. Following DeepseekCoder (Guo et al., 2024) and Code LLaMA (Rozière et al., 2023), StarCoder2 use Rotary Positional Encodings. We evaluated StarCoder2-{7B, 15B} on our EvoCodeBench, which was trained on over 3.5 trillion tokens in 17 programming languages from Stack v2.

- **Gemma** (GemmaTeam, 2024) is a lightweight open model family built from the research and technology used to create Gemini models. Gemma models demonstrate strong performance on academic benchmarks in language understanding, reasoning, and security. Gemma releases models in two sizes (2 billion and 7 billion parameters) and provides pre-training and fine-tuning checkpoints. Gemma is released by Google DeepMind[9] on February 21, 2024, based on the transformer decoder. Models are trained on a context length of 8192 tokens. We evaluated Gemma-7b in our EvoCodeBench performance.

- **Qwen** (Bai et al., 2023) is a comprehensive language model series containing different models with different parameters, including basic pre-trained language models and chat models fine-tuned through human alignment technology. The Qwen series is open-sourced including Qwen-1.8B, Qwen-7B, Qwen-14B and Qwen-72B, and the corresponding Qwen-Chat series, which the Alibaba Group released on August 3, 2023. Qwen has conducted stable pre-training on multi-lingual data of up to 3 trillion tokens, covering fields, languages (focusing on Chinese and English), etc. QWEN is designed using a modified version of the Transformer architecture, specifically adopting the open-source approach of LLaMA (Touvron et al., 2023). We evaluated the performance of Qwen1.5-7b on our EvoCodeBench.

[8] https://www.bigcode-project.org/

[9] https://deepmind.google/

Please complete the *{function_name}* function in the given Python code.

Input Code:
```Python
{signature}
{requirement}
```
Completed Code:

Figure 7: The prompt template in the without context setting.

Please complete the *{function_name}* function based on the contexts above the function.

The contexts above the function are:
```Python
{contexts_above}
```

The code to be completed is:
```Python
{signature}
{requirement}
```
Completed code:

Figure 8: The prompt template in the local file (completion) setting.

B.2 Prompt Templates

The prompt templates used for instruction-tuning models (*i.e.,* gpt-4 and gpt-3.5) are shown in Figure 7, 8, 9, and 10. `{function_name}`, `{contexts_above}`, `{contexts_below}`, `{signature}`, and `{requirement}` are placeholders.

For other standard language models, the prompt templates are shown as follows: ① Without context: [`signature; requirement`]; ② Local file (completion): [`context_above; signature; requirement`]; ③ Local file (infilling): [`prefix_id; context_above; signature; requirement; suffix_id; context_below; middle_id`]. Where [`;`] denotes the concatenation operation of strings. `{prefix_id}`, `{suffix_id}`, `{middle_id}` are special tokens used in code infilling. For different LLMs, we reuse their official special tokens to make prompts.

Please complete the *{function_name}* function in the middle of a file.

The contexts above the function are:
```Python
{contexts_above}
```
The contexts below the function are:
```Python
{contexts_below}
```

The code to be completed is:
```Python
{signature}
{requirement}
```
Completed code:

Figure 9: The prompt template in the local file (infilling) setting.

Please complete the *{function_name}* function based on some functions with similar names.

The functions with similar names are:
```Python
{similar_functions}
```

The code to be completed is:
```Python
{signature}
{requirement}
```
Completed code:

Figure 10: The prompt template in the similar function setting.